

Mathematical Report

Optimal Patrol Path for Security Robot

Introduction

This report outlines the mathematical and algorithmic foundation for determining the shortest simple closed polygonal path that encloses N axis aligned rectangular machines on a factory floor. The optimal solution relies on transforming the geometric rectangles into a set of discrete points and computing their convex hull.

1 Mathematical Explanation

1.1 (a) Transforming the Rectangle Problem into a Point Problem

Every rectangle is defined by two opposite corners. (x_1, y_1) and (x_2, y_2) From those, you can reconstruct all four corners. (x_1, y_1) , (x_2, y_1) , (x_1, y_2) , (x_2, y_2)

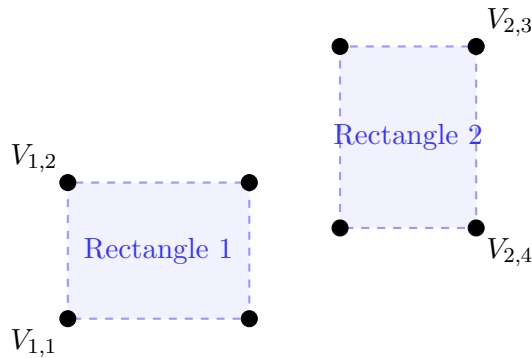
So for each rectangle, you add its four corners into a single set of points. After processing all N rectangles, you'll have $4N$ points.

$$V_{i,1} = (x_{i,1}, y_{i,1})$$

$$V_{i,2} = (x_{i,1}, y_{i,2})$$

$$V_{i,3} = (x_{i,2}, y_{i,2})$$

$V_{i,4} = (x_{i,2}, y_{i,1})$ By extracting these vertices for all N rectangles, we generate a unified, flattened set S of $K = 4N$ points. At this stage, we disregard the boundaries and interiors of the rectangles entirely. The problem simplifies to finding the boundary that encloses this point set S .



1.2 (b) Validity of the Convex Hull as the Optimal Path

Think about what the patrol path is actually supposed to achieve. The goal is to create a loop that goes around every rectangle on the floor. Since each rectangle is completely defined by its corner points, if we can enclose all of those corners, then the entire rectangles will automatically be enclosed as well.

So, instead of thinking about rectangles, we can simplify the problem into something much cleaner:

“What is the shortest possible loop that surrounds all the corner points?”

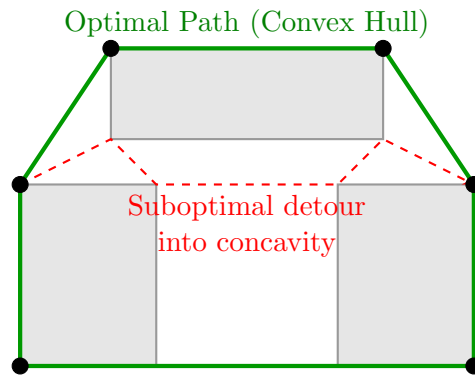
That shortest enclosing loop is called the convex hull.

The reason this works comes from an important geometric idea. The convex hull is the smallest convex shape that can contain all the given points. Imagine stretching a rubber band around the outermost corners, the shape the rubber band forms is the convex hull.

Now, suppose the path bent inward somewhere, creating a zig-zag shape. That inward bend would actually make the path longer than necessary. If you straightened that section, the loop would become shorter while still enclosing all the points. Because of this, any shortest possible boundary must be convex, which is exactly why the convex hull gives the minimum perimeter.

This also works perfectly for the rectangles themselves in practice

Once all the corner points are inside the hull, the full rectangles are guaranteed to be inside too, because rectangles are convex shapes. The hull boundary is made only from straight lines connecting the outermost points. Since the rectangles may touch but do not overlap, these boundary lines naturally trace around the outside without cutting through any rectangle interiors.



1.3 (c) Algorithm Description: Andrew’s Monotone Chain

To efficiently compute the convex hull of all $4N$ corner points, we use Andrew’s Monotone Chain Algorithm. The idea behind this algorithm is to gradually build the outer boundary of the point set by constructing two parts: the lower hull and the upper hull.

- Sorting the Points:

The first step is to sort all points lexicographically, meaning we sort primarily by the x -coordinate and then by the y -coordinate if two points share the same x value. This sorting step is important because it allows us to process the points in a consistent left-to-right order. Once the points are ordered, we can build the hull step by step while only checking the most recently added points for any violations of convexity.

- Building the Hull Using Orientation Tests:

While constructing the hull, we maintain a list (or stack) of points that currently form the boundary. As each new point is considered, we look at the last two points already on the hull together with the new point. Using the cross product (orientation test), we determine whether these three points form a left turn or a right turn.

If they form a right turn, or if the points are collinear, it means the middle point cannot be part of the convex boundary because it creates an inward bend or unnecessary edge. In that case, we remove the middle point from the hull. This process continues until the last three points form a

proper left turn, which preserves the convex shape. The same procedure is performed once from left to right to build the lower hull, and again from right to left to build the upper hull.

- Time Complexity:

For N rectangles, there are $K = 4N$ corner points in total.

- Sorting the K points requires $O(K \log K)$ time.
- During hull construction, each point is added and removed at most once for the lower hull and once for the upper hull, so this phase runs in linear time, $O(K)$.
- Therefore, the total running time is dominated by the sorting step:

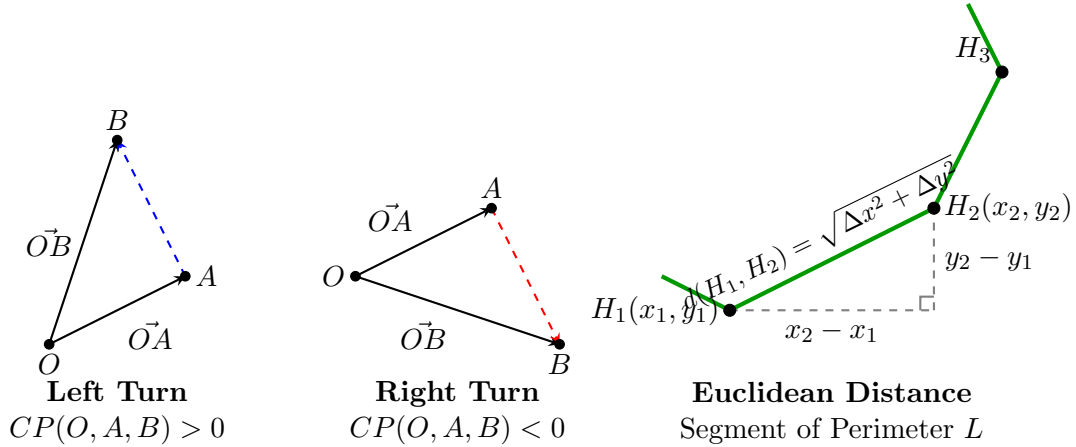
$$O(4N \log 4N)$$

which simplifies to:

$$\mathcal{O}(N \log N)$$

2 Mathematical Details (Formulas)

The algorithm relies on the following precise mathematical formulations to evaluate geometry and calculate the final optimal path length.



2.1 Orientation Test and the Cross Product

To determine whether three points $O(x_o, y_o)$, $A(x_a, y_a)$, and $B(x_b, y_b)$ constitute a left turn, right turn, or are collinear, we use the 2D cross product of the vectors $\vec{OA}$ and $\vec{OB}$.

The scalar value of the 2D cross product is given by the formula:

$$CP(O, A, B) = (x_a - x_o)(y_b - y_o) - (y_a - y_o)(x_b - x_o) \quad (1)$$

The orientation is determined by the sign of CP :

- If $CP > 0$: The sequence $O \rightarrow A \rightarrow B$ makes a strictly **counter-clockwise (left) turn**.
- If $CP < 0$: The sequence $O \rightarrow A \rightarrow B$ makes a strictly **clockwise (right) turn**.
- If $CP = 0$: The points O , A , and B are **collinear**.

During the Monotone Chain algorithm, we pop points from the stack as long as $CP(O, A, B) \leq 0$ to strictly enforce convexity and remove redundant collinear points on the boundary.

2.2 Euclidean Distance

Once the convex hull vertices are identified, we need to calculate the distance between adjacent vertices on the hull. The straight-line distance d between any two points $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$ on a 2D Cartesian plane is given by the standard Euclidean distance formula:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (2)$$

2.3 Perimeter of the Convex Hull

Suppose the convex hull is represented as an ordered sequence of M vertices $H = \{H_1, H_2, \dots, H_M\}$ arranged in counter-clockwise order around the perimeter.

The total minimum path length L (the perimeter of the convex hull) is the sum of the Euclidean distances of all consecutive line segments, including the segment connecting the last vertex back to the first vertex. The formula is:

$$L = \sum_{j=1}^M d(H_j, H_{j+1}) \quad (3)$$

where we conventionally define $H_{M+1} \equiv H_1$ to close the loop.